

ShopOS: Complete Architecture, PRD, and Operational Manual

1. Executive Summary

ShopOS is a specialized, highly resilient enterprise resource planning (ERP) and Point of Sale (POS) system designed specifically for auto parts retail and logistics. Combining a React/Vite/TypeScript frontend with a Node.js/Express backend—and utilizing Google Sheets as a highly accessible NoSQL data store—ShopOS bridges the gap between traditional retail environments and modern web applications.

A defining feature of ShopOS is its **Offline-First Hybrid Architecture**, ensuring that storefront operations (Counter sales, inventory lookups) continue seamlessly even during internet outages, gracefully synchronizing data when connectivity is restored.

2. Product Requirements Document (PRD) & Project Scope

2.1 Problem Statement

Auto parts shops frequently struggle with expensive, proprietary ERPs that are difficult to customize and expensive to scale. Alternatively, shops use disconnected spreadsheets that lack real-time synchronization, auditing capability, and role-based security. Power outages and internet instability often halt business operations entirely.

2.2 Target Audience & User Personas

- Administrators (Owners/Managers):** Need high-level analytics (Recharts), complete audit trails, user management, and the ability to define global pricing parameters (e.g., AED to KES conversion rates).
- Counter Staff (Sales):** Require an incredibly fast, responsive POS interface to look up parts quickly, process cash/M-Pesa/Credit transactions, and provide immediate service to customers.
- Warehouse Workers:** Need mobile-accessible interfaces to log incoming shipments, update exact stock quantities, and verify part locations without dealing with financial data.

2.3 Scope Definition

In-Scope:

- Real-time inventory tracking and mutation.
- Point of Sale (POS) cart system for processing multi-item transactions.
- Offline support and automated background synchronization.
- Role-Based Access Control (RBAC) with secure session management.
- Comprehensive audit trails capturing all destructive actions.
- Analytics dashboard analyzing sales volume and inventory health.
- Dynamic backend synchronization using Google Sheets API.

Out-of-Scope (for Current Version):

- Integration with external accounting software (e.g., QuickBooks).
 - Direct e-commerce integration (customer-facing storefront).
 - Automated hardware integration (e.g., direct barcode scanner drivers, though standard keyboard-wedge scanners work natively).
-

3. System Architecture

The application employs a decoupled client-server model enabling independent scaling.

3.1 Frontend (Client-Side)

- **Framework:** React 19 + TypeScript + Vite. Ensures type safety, incredibly fast build times, and an optimized production bundle.
- **UI & Styling:** Tailwind CSS for a utility-first, fully responsive design system. Includes `lucide-react` for iconography.
- **Routing:** React Router v7 providing Single Page Application (SPA) navigation without page reloads.
- **Data Visualization:** `recharts` for rendering complex financial dashboards.
- **State Management & Offline Storage:**
 - Custom `SafeStorage` wrapper around `window.localStorage` that falls back to in-memory maps if storage is restricted (e.g., incognito mode).
 - Custom `SyncQueue` : Intercepts non-GET HTTP requests during network outages, stores the exact HTTP payload (POST/PUT/DELETE), and replays them chronologically upon reconnecting to the internet.

3.2 Backend (API Service)

- **Framework:** Node.js + Express.js
- **Database Engine:** Google Sheets API. A wildly unconventional but highly effective approach for this domain. It uses specific tabs/worksheets as database tables: `INVENTORY` , `USERS` , `SESSIONS` , `SALES` , `SETTINGS` , `AUDIT_LOG` .
- **Identity & Security:**
 - Standard JSON Web Tokens (JWT) are bypassed in favor of a **Custom Cryptographic Session Pattern**.
 - Secure session tokens are generated, stored in the `SESSIONS` sheet with expiry limits, and checked via lightweight middleware. Allows for immediate remote revocation of compromised devices.

3.3 The Data Pipeline & Synchronization Flow

1. **Read Request:** Frontend calls `GET /api/inventory` . If online, it fetches from Express. Express queries Google Sheets, formats the JSON, and returns it. Frontend updates its local cache. If offline, frontend immediately falls back to local cache.
 2. **Write Request:** Frontend calls `POST /api/sales` .
 - *Online:* Hits backend, updates `SALES` and decrements `INVENTORY` in Google Sheets concurrently.
 - *Offline:* Frontend queues the request in `SYNC_QUEUE` , updates the local UI optimistically (cache sales, decrement cache inventory), and waits.
 3. **Reconnection:** `window.addEventListener('online')` fires. Frontend flushes `SYNC_QUEUE` synchronously to the backend to ensure eventual consistency.
-

4. Comprehensive Feature Manual

4.1 Authentication & User Modes

- **Login Flow:** Users enter credentials at `/login` . The system securely stores a `session_id` mapping to their hardware/browser footprint.

- **Roles:**
 - **Admin** : Full read/write access everywhere including Settings, Audit Logs, and User Management.
 - **Counter** : Can read inventory, process sales, view daily sales history. Cannot modify settings or delete logs.
 - **Worker** : Can read inventory, update stock counts. Cannot view high-level financial reports or alter sales records.

4.2 Inventory Management Module

- **Item Schema:** Every part has a `uuid` , `part_number` , `name` , `tags` , `make` (Genuine/Japan/Aftermarket), `aed_buying_price` , `selling_price` , `stock_qty` , and `min_stock` .
- **Dynamic Pricing:** The system globally maps AED (UAE Dirham) buying prices to local KES (Kenyan Shilling) selling prices using global settings (Conversion overhead percentages and exchange rates).
- **CRUD Operations:** Administrators can add, edit, or delete parts.
- **Bulk Imports:** The system supports importing massive CSV/JSON arrays of parts to handle restocks effectively.
- **Low Stock Alerts:** Items dipping below `min_stock` trigger UI alerts and appear in dedicated filtered views.

4.3 Point of Sale (POS) / Sales Module

- **Cart System:** Counter staff search for parts (by name, tag, or part number), add them to the cart, specify quantities, and apply bulk operations.
- **Checkout:** Transactions require a Payment Method (Cash, M-Pesa, Credit). Once processed, the system assigns a unique `batch_id` (e.g., `RCPT-XYZ123`).
- **Inventory Reconciliation:** A successful sale automatically subtracts the `stock_qty` from the active inventory.

4.4 Settings & Configuration

Accessed strictly by Admins. Allows on-the-fly modification of:

- `aed_rate` : Current exchange rate. Immediately updates expected baselines across the entire inventory UI.
- `conversion_percent` : Standard import duty/logistics overhead markup.
- `default_min_stock` : The default fallback for low-stock triggers.

4.5 Reporting & Analytics

- **Sales Summary:** View chronological revenue metrics separated by payment method. Filterable by date ranges.
- **Inventory Health:** Total calculated valuation of all stock currently sitting in the warehouse. Identifies dead stock vs. fast-moving items.

5. Scale Optimization Strategies

While the current architecture (Google Sheets DBMS) is wildly efficient for zero-cost operation and high accessibility, it introduces specific scaling bottlenecks (rate limits, transaction locks).

5.1 Immediate Backend Optimizations

- **Implementation of Redis/Memcached:** Place a powerful caching layer in front of the Google Sheets API to reduce read-cycles. Use a write-through cache strategy.
- **Message Broker (RabbitMQ / Kafka):** If 5+ counter terminals are submitting sales simultaneously, directly hitting Google Sheets will cause write-locks or HTTP 429 Too Many Requests. A centralized queue on the Node.js layer should sequence and batch write operations.

5.2 Database Migration Path (The Ultimate Optimization)

- Currently, the transition is inevitable if the business surpasses ~50,000 SKUs or 1,000 daily transactions.
- **Target Architecture:** Migrate from Google Sheets to **PostgreSQL**.
- **Migration Plan:** The backend codebase is decoupled. The `services/sheets.js` logic can be hot-swapped for a `services/pg.js` module running parameterized SQL queries without requiring *any* frontend changes.

5.3 Frontend Optimizations

- **Virtualization / Windowing:** If the inventory list grows to 10,000+ items, the React DOM will lag. Implement `react-window` or `react-virtuoso` in the Inventory tables to only render the 30 items visible on screen.
- **Service Workers (PWA):** Upgrade the current `SafeStorage` offline mechanism to a full Progressive Web App utilizing standard Service Workers and IndexedDB. This allows the caching of the actual React HTML/JS bundles for true zero-network loading.

6. Future Upgrades & Product Roadmap

6.1 Phase 1: Operational Enhancements (Next 6 Months)

- **Barcode Scanning Support:** Native integration to accept USB keyboard-wedge barcode scanner inputs directly into the POS search bar.
- **Receipt Generation & Printing:** Auto-generate PDF receipts (`jsPDF`) and send print commands directly to ESC/POS thermal receipt printers via the browser's WebUSB or Bluetooth APIs.
- **Customer Relationship Management (CRM):** Track credit sales tied to specific VIP customer accounts, track outstanding balances, and send automated M-Pesa payment reminders (via third-party API like Daraja).

6.2 Phase 2: Analytics & Supply Chain (6-12 Months)

- **Predictive Restocking AI:** Analyze historical sales velocity per part number to automatically generate "Suggested Purchase Orders" for suppliers in Dubai/Japan before stock runs out.
- **Multi-Branch Support:** Alter the database schema to track `branch_id` . Allow transfers of inventory between different physical store locations.

6.3 Phase 3: E-Commerce & Customer Ecosystem (1 Year+)

- **Public B2B Portal:** A read-only, authenticated frontend where frequent mechanics/garages can log in, check if ShopOS has exactly the part they need in stock, and reserve it.
- **Automated WhatsApp Integrations:** When an out-of-stock part requested by a customer arrives via import, automatically fire a WhatsApp message (via Twilio/Meta API) to the customer.

7. Operational & Technical Troubleshooting Guide

7.1 Backend Not Starting (`ERR_REQUIRE_ESM`)

- **Cause:** The backend is configured as an ES Module (`"type": "module"` in `package.json`), but legacy CommonJS `require()` is being attempted.
- **Fix:** Ensure all imports use the `import x from 'y'` syntax and file extensions (e.g., `import auth from './middleware/auth.js'`).

7.2 Google Sheets Connection Failure / Write Errors

- **Cause:** Service account permissions revoked, or Google API Rate Limit Exceeded (Quota limits).
- **Fix:** Verify `GOOGLE_PRIVATE_KEY` formatting (must contain literal `\n` characters, not actual line breaks if set in pure bash environments). If rate limited, implement exponential backoff retry logic (already present in the frontend, might need reinforcement on the backend-to-Google pipeline).

7.3 Ghost Sales (Inventory mismatch)

- **Cause:** An offline sale was made, but the browser's `localStorage` was aggressively cleared by the user or OS before internet reconnection.
- **Fix:** Educate counter staff never to clear browser data mid-shift. Long-term fix requires shifting from `localStorage` to `IndexedDB` which is significantly more persistent.

End of Documentation